



K.R. Mangalam University

School of Engineering & Technology

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

Program: BCA (AI & DS)

Course Code - ENCA301 (2025-2026)

Submitted by:

Name: Bhavya Rattan

Roll Number: 2401201004

Course: BCA (AI & DS) - Section B

Submitted To:

Dr. Aarti Sangwan

Assistant Professor

Index

S. No	Topic	Description
1	Introduction	Overview of algorithm analysis and objectives of the experimental study
2	Experimental Design	System environment, benchmarking procedure, datasets, and performance metrics
3	Algorithm Implementations	Working principles, pseudocode, Python implementations, and complexity of sorting and Fibonacci algorithms
4	Experimental Results	Recorded execution time, comparison counts, and memory consumption
5	Complexity Comparison	Theoretical complexity versus observed empirical behavior
6	Graphical Analysis	Interpretation of execution-time, memory, and operation-count graphs
7	Discussion	Scalability analysis, theoretical observations, and engineering trade-offs
8	Conclusion	Documentation, and tools used

1. Introduction

1.1 Background

- Algorithms form the foundation of computer science because they define the sequence of steps used to solve computational problems. However, producing the correct output is not the only requirement of a good algorithm. As the size of the input increases, the algorithm must also remain efficient in terms of execution time and memory consumption.
 - The Design and Analysis of Algorithms laboratory provides an opportunity to study this difference experimentally. Theoretical complexity analysis predicts how an algorithm grows as the input size increases, while empirical benchmarking measures how the algorithm actually behaves in a real execution environment.
-

1.2 Need for Experimental Algorithm Analysis

Asymptotic notation such as $O(n)$, $O(n^2)$, and $O(n \log n)$ describes the growth rate of an algorithm. However, theoretical analysis alone does not show the complete practical behaviour of an implementation.

Actual performance can also depend on:

- Input distribution.
- Number of comparisons and operations.
- Memory allocation.
- Recursive function overhead.
- Constant factors.
- Programming language implementation.
- Hardware and operating system behaviour.

Therefore, experimental analysis is necessary to validate theoretical predictions.

For example, at an input size of $n = 10,000$ in the recorded experiment, Bubble Sort required approximately **13.57 seconds**, while Quick Sort completed in approximately **0.0158 seconds**. Although both algorithms perform the same task, their scalability differs significantly. Similarly, for Fibonacci computation at $n = 40$, the Naive Recursive approach required approximately **27.02 seconds**, whereas the Iterative approach required only approximately **0.0000059 seconds**.

2. EXPERIMENTAL DESIGN AND BENCHMARKING METHODOLOGY

2.1 Execution Environment

All algorithms were implemented and experimentally evaluated using Python. The experiments were conducted under the following software environment:

Component	Configuration
Programming Language	Python 3.14
Operating System	Windows 11
System Architecture	x86_64
Execution Timer	time.perf_counter()
Memory Profiler	tracemalloc
Random Seed	2026
Number of Executions	3 per experiment
Final Timing Value	Median of 3 executions

2.2 Performance Metrics

Three primary metrics were used for sorting analysis.

Execution Time Execution time was measured using- `time.perf_counter()`

This function provides a high-resolution wall-clock timer suitable for benchmarking.

Each experiment was executed three times, and the median execution time was recorded.

Comparison Count The number of element comparisons performed by each sorting algorithm

This metric helps explain why two algorithms with different complexity classes behave differently as input size increases.

Peak Memory Consumption Peak heap memory allocation was measured using -`tracemalloc`

Memory values were recorded in kilobytes. This measurement was particularly important for comparing Merge Sort, which requires auxiliary arrays, with the in-place sorting approaches.

3. ALGORITHM IMPLEMENTATION

3.1 Bubble Sort

Working Principle

Bubble Sort repeatedly compares adjacent elements. If two neighbouring elements are in the wrong order, they are swapped.

After each complete pass, the largest unsorted element moves towards the end of the list.

Pseudocode:

```
ALGORITHM BubbleSort(A, n)

FOR i = 0 TO n - 2
    swapped = FALSE

    FOR j = 0 TO n - 2 - i
        IF A[j] > A[j + 1]
            SWAP A[j], A[j + 1]
            swapped = TRUE
        END IF
    END FOR

    IF swapped == FALSE
        BREAK
    END IF
END FOR

RETURN A
```

Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Auxiliary Space	$O(1)$

3.2 Insertion Sort

Working Principle

Insertion Sort gradually constructs a sorted portion of the array.

Each new element is compared with the already sorted elements and inserted into its correct position.

Pseudocode:

```

ALGORITHM InsertionSort(A, n)

FOR i = 1 TO n - 1
    key = A[i]
    j = i - 1

    WHILE j >= 0 AND A[j] > key
        A[j + 1] = A[j]
        j = j - 1
    END WHILE

    A[j + 1] = key
END FOR

RETURN A

```

Complexity

Case	Time Complexity
Best Case	$O(n)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Auxiliary Space	$O(1)$

3.3 Merge Sort

Working Principle

Merge Sort uses the divide-and-conquer technique.

The array is repeatedly divided into smaller subarrays until individual elements remain. These subarrays are then merged in sorted order.

Pseudocode:

```

ALGORITHM MergeSort(A, left, right)

IF left < right
    mid = (left + right) / 2

    MergeSort(A, left, mid)
    MergeSort(A, mid + 1, right)

    Merge(A, left, mid, right)

```

Complexity

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Auxiliary Space	$O(n)$

3.4 Quick Sort

Working Principle

Quick Sort selects a pivot element and partitions the remaining elements around it.

Elements smaller than the pivot are placed on one side, while larger elements are placed on the other side.

The same process is recursively applied to the resulting partitions.

Pseudocode:

```
ALGORITHM QuickSort(A, low, high)

IF low < high
    p = Partition(A, low, high)

    QuickSort(A, low, p - 1)
    QuickSort(A, p + 1, high)
```

Complexity

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$
Auxiliary Space	$O(\log n)$

3.5 NAIVE RECURSIVE FIBONACCI

Pseudocode:

```
ALGORITHM Fib_Recursive(n)

IF n <= 1
    RETURN n
END IF

RETURN Fib_Recursive(n - 1) + Fib_Recursive(n - 2)
```

Complexity

$$\begin{aligned} \text{Time} &= O(2^n) \\ \text{Space} &= O(n) \end{aligned}$$

3.6 ITERATIVE FIBONACCI

Pseudocode:

```
ALGORITHM Fib_Iterative(n)

IF n <= 1
    RETURN n
END IF

a = 0
b = 1

FOR i = 2 TO n
    c = a + b
    a = b
    b = c
END FOR

RETURN b
```

The algorithm stores only the two most recent Fibonacci values.

Complexity

$Time = O(n)$
 $Space = O(1)$

3.7 DYNAMIC PROGRAMMING FIBONACCI

Pseudocode:

```
ALGORITHM Fib_DP(n, memo)

IF n IN memo
    RETURN memo[n]
END IF

IF n <= 1
    RETURN n
END IF

memo[n] = Fib_DP(n - 1, memo) + Fib_DP(n - 2, memo)

RETURN memo[n]
```

Complexity

$Time = O(n)$
 $Space = O(n)$

4. Experimental Results

4.1 Execution Time

Input Size n	Bubble Sort (s)	Insertion Sort (s)	Merge Sort (s)	Quick Sort (s)
80	0.0008210	0.0004461	0.0004190	0.0001265
500	0.0234579	0.0117817	0.0026053	0.0012329
1,200	0.1632262	0.0709302	0.0075427	0.0036252
3,000	1.1424957	0.5462212	0.0238204	0.0114098
10,000	13.5684000	5.8821900	0.0354120	0.0158210

4.2 Effect of Input Distribution

Algorithm	Sorted (s)	Reverse (s)	Random (s)
Bubble Sort	0.0000092	0.0009074	0.0008210
Insertion Sort	0.0000122	0.0007832	0.0004461
Merge Sort	0.0007734	0.0003775	0.0004190
Quick Sort	0.0005268	0.0001418	0.0001265

4.3 Comparison Count Analysis

Table 4.3: Number of Comparisons on Random Input

Input Size n	Bubble Sort	Insertion Sort	Merge Sort	Quick Sort
80	3,139	1,872	410	435
500	124,497	67,533	3,844	4,569
1,200	719,364	372,245	10,748	13,165
3,000	4,498,464	2,321,702	30,874	39,267
10,000	49,977,294	25,183,912	120,488	148,932

4.4 Memory Consumption Analysis

Table 4.4: Peak Memory Consumption on Random

Input Size n	Bubble Sort (KB)	Insertion Sort (KB)	Merge Sort (KB)	Quick Sort (KB)	Input Size n
80	0.68	0.68	2.13	0.69	80
500	3.93	3.93	12.20	3.94	500
1,200	9.40	9.40	29.64	9.41	1,200
3,000	23.46	23.46	73.60	23.47	3,000
10,000	78.18	78.18	270.45	78.20	10,000

4.5 Fibonacci Algorithm Performance

The Fibonacci algorithms were evaluated increasing values of n .

Value n	Naive Recursive (s)	Iterative (s)	DP Memoization (s)
8	0.0000069	0.0000026	0.0000071
20	0.0019647	0.0000037	0.0000129
32	0.6438540	0.0000052	0.0000179
36	4.4501978	0.0000056	0.0000200
40	27.0234541	0.0000059	0.0000207

4.5 Fibonacci Memory Consumption

Value n	Naive Recursive (KB)	Iterative (KB)	DP Memoization (KB)
8	0.00	0.00	0.46
20	0.02	0.02	0.86
32	0.02	0.02	1.66
36	0.02	0.02	1.66
40	0.02	0.02	1.66